MANUAL DE IMPLEMENTACIÓN · WINDOWS

Spring Cloud Config Server con tres microservicios REST

Guía reproducible desde cero, sin Docker ni bases de datos. Se integra un repositorio Git local real; GitHub queda como alternativa opcional.

Componente	Función	Puerto
Config Server	Lee YAML desde Git y sirve configuración	8888
Usuarios	API REST /mensaje	8081
Productos	API REST /mensaje	8082
Pedidos	API REST /mensaje	8083

1. Requisitos y versiones

JDK 17 o posterior, Git y PowerShell. Se incluyen Maven Wrapper y los cuatro proyectos Maven. La primera compilación necesita acceso a Maven Central para descargar el wrapper, Maven y las dependencias.

Tecnología	Versión fijada
Java	17 (compatible con tu OpenJDK 17)
Spring Boot	3.5.15
Spring Cloud	2025.0.3
Apache Maven	3.9.9, descargado por Maven Wrapper

2. Descomprimir y abrir PowerShell

Extraer SpringCloudConfig_ProyectoCompleto.zip en una ruta corta, por ejemplo C:\SpringCloudConfig. Entrar a la carpeta que contiene scripts y config-repo, nunca ejecutar desde C:\Windows\System32.

```
cd C:\SpringCloudConfig
java -version
javac -version
git --version
```

Si javac no existe, se instaló únicamente un runtime (JRE); necesitas un JDK.

3. Inicializar Git y compilar

El primer script crea un repositorio Git en config-repo, usa la rama main, realiza el commit inicial y compila los cuatro proyectos. No necesita que configures una cuenta GitHub.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass `
  -File .\scripts\01-Preparar.ps1
```

Durante la primera descarga es necesario internet. Espera a ver el mensaje PREPARACION COMPLETA. Si aparece un error de Maven, registra el log y comprueba la conexión.

4. Arrancar los cuatro procesos

El segundo script calcula la ruta del repositorio Git para Windows, inicia cada JAR en un proceso independiente y comprueba la salud de cada servicio. Guarda logs y PID para facilitar la parada.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass `
  -File .\scripts\02-Iniciar.ps1
```

5. Verificar respuestas

```
powershell.exe -NoProfile -ExecutionPolicy Bypass `
  -File .\scripts\03-Verificar.ps1
```

Método	URL	Qué comprueba
GET	http://localhost:8888/usuarios/default	Config Server lee Git
GET	http://localhost:8081/mensaje	Usuarios usa su YAML
GET	http://localhost:8082/mensaje	Productos usa su YAML
GET	http://localhost:8083/mensaje	Pedidos usa su YAML

En Postman selecciona GET y copia cada URL. Para Config Server se verá el objeto JSON con propertySources; en las API /mensaje, los campos servicio, mensaje y version. Son respuestas esperadas, no capturas simuladas.

6. Prueba clave: Git cambia la API

Este comando modifica solo config-repo/usuarios.yml, crea un commit Git, invoca el endpoint de refresco y consulta el nuevo valor de /mensaje. No edita ni recompila la clase Java.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass `
  -File .\scripts\04-Cambiar-Mensaje.ps1 `
  -NuevoMensaje "Cambio visible en clase"
```

Verifica el nuevo commit y el JSON de respuesta:

```
git -C .\config-repo log --oneline -3
Invoke-RestMethod http://localhost:8081/mensaje
```

Si se hace manualmente en Postman, primero edita usuarios.yml y realiza git add + git commit. Después envía:

```
POST http://localhost:8081/actuator/refresh
GET  http://localhost:8081/mensaje
```

7. Detener procesos

```
powershell.exe -NoProfile -ExecutionPolicy Bypass `
  -File .\scripts\05-Detener.ps1
```

8. Estructura del proyecto

```
config-server/           # Spring Cloud Config Server
microservicio-usuarios/  # Cliente 1, API /mensaje
microservicio-productos/ # Cliente 2, API /mensaje
microservicio-pedidos/   # Cliente 3, API /mensaje
config-repo/             # Archivos .yml versionados con Git
scripts/                 # Preparar, iniciar, verificar, cambiar y parar
presentacion/            # Presentación PowerPoint de 8 diapositivas
manual/                  # Esta guía PDF
```

La inicialización inicial de config-repo produce su carpeta .git solo en la computadora donde se ejecute.

9. Evidencias para la exposición

Tomar capturas reales durante el ensayo y, si el profesor lo exige, anexarlas después a esta guía. No se han fabricado pruebas de ejecución.

Captura	Lo que debe visualizarse
A	Config Server respondiendo /usuarios/default
B	API Usuarios, Productos y Pedidos en Postman o navegador
C	Git log antes y después del commit
D	POST /actuator/refresh en Postman
E	GET /mensaje con el texto actualizado

10. Problemas frecuentes

Síntoma	Comprobación recomendada
Java/Maven no compila	Verificar javac -version y red para Maven Central
Config Server devuelve error	Revisar Git main, commit inicial, archivo YAML y logs
Un servicio no inicia	Asegurar que el puerto 8888 esté disponible antes
Cambios no se reflejan	Confirmar git commit y luego POST a /actuator/refresh
Puerto ocupado	Detener versiones previas y repetir arranque

11. Consideraciones de seguridad

La práctica limita el acceso a 127.0.0.1. El endpoint /actuator/refresh no debe exponerse públicamente sin autenticación y controles de red. Nunca almacenar contraseñas reales en un repositorio abierto.

12. Fuentes técnicas

Spring Cloud: spring.io/projects/spring-cloud/

Config Client: docs.spring.io/spring-cloud-config/reference/client.html

Git backend: docs.spring.io/spring-cloud-config/reference/server/environment-repository/git-backend.html

Spring Boot 3.5: docs.spring.io/spring-boot/3.5/system-requirements.html

Estado de verificación: se validó el esquema de archivos, las rutas y los YAML durante la generación. La compilación y prueba HTTP final dependen de ejecutarlos en Windows; documentar evidencias auténticas tras el ensayo.